

Informe de Práctica: Implementación de Servicio DNS RESTful

1. Introducción

El objetivo de esta práctica ha sido la transición de un modelo de computación distribuida basado en RMI (Remote Method Invocation) a un modelo RESTful (Representational State Transfer) utilizando el framework FastAPI. A diferencia de RMI, que utiliza serialización binaria opaca, el uso de REST e HTTP permite una mayor interoperabilidad, visibilidad del tráfico y escalabilidad del servicio.

2. Diseño del Servicio

El servicio se ha implementado mediante una arquitectura cliente-servidor donde:

- Protocolo: HTTP/1.1 sobre TCP.
- Formato de datos: JSON (JavaScript Object Notation), garantizando la estandarización y legibilidad.
- End-points:
 - POST /dns: Registro de nuevas entradas DNS.
 - GET /dns/{nombre}: Consulta de resolución de nombres.

3. Evidencias de Ejecución

A continuación, se presenta la validación del protocolo mediante análisis de tráfico con Wireshark.

A. Operación de Escritura (POST)

Se ha validado la creación de registros mediante el método POST.

- Acción: Envío de payload JSON con los campos nombre, ip y ttl.
- Resultado: Recepción de código 201 Created, confirmando el persistido del recurso en el servidor.

*Adapter for loopback traffic capture

Archivo Edición Visualización Ir Captura Analizar Estadísticas Telefonía Wireless Herramientas Ayuda

tcp.port == 8000

No.	Time	Source	Destination	Protocol	Length	Info
5	6.036216400	127.0.0.1	127.0.0.1	TCP	44	53271 → 8000 [FIN, ACK] Seq=1 Ack=1
6	6.036264700	127.0.0.1	127.0.0.1	TCP	44	8000 → 53271 [ACK] Seq=1 Ack=2 Win=0
7	6.036857000	127.0.0.1	127.0.0.1	TCP	56	49609 → 8000 [SYN] Seq=0 Win=65535
8	6.036926300	127.0.0.1	127.0.0.1	TCP	56	8000 → 49609 [SYN, ACK] Seq=0 Ack=1
9	6.036948600	127.0.0.1	127.0.0.1	TCP	44	49609 → 8000 [ACK] Seq=1 Ack=1 Win=0
10	6.037183400	127.0.0.1	127.0.0.1	HTTP/1.1	754	POST /dns HTTP/1.1, JSON (application/json)
11	6.037200200	127.0.0.1	127.0.0.1	TCP	44	8000 → 49609 [ACK] Seq=1 Ack=711
14	6.039103600	127.0.0.1	127.0.0.1	TCP	174	8000 → 49609 [PSH, ACK] Seq=1 Ack=1
15	6.039130600	127.0.0.1	127.0.0.1	TCP	44	49609 → 8000 [ACK] Seq=711 Ack=1
16	6.039196300	127.0.0.1	127.0.0.1	HTTP/1.1	75	HTTP/1.1 201 Created, JSON (application/json)
17	6.039208700	127.0.0.1	127.0.0.1	TCP	44	49609 → 8000 [ACK] Seq=711 Ack=1

B. Operación de Lectura (GET)

Se ha verificado la recuperación de datos mediante el método GET.

- Acción: Solicitud del recurso mediante path parameter.
- Resultado: Recepción de código 200 OK con el JSON solicitado.

2	0.000064400	127.0.0.1	127.0.0.1	TCP	56	8000 → 65262 [SYN, ACK] Seq=0 Ack=1
3	0.000087100	127.0.0.1	127.0.0.1	TCP	44	65262 → 8000 [ACK] Seq=1 Ack=1 Win=0
4	0.000282100	127.0.0.1	127.0.0.1	HTTP	618	GET /dns/hola.com HTTP/1.1
5	0.000298100	127.0.0.1	127.0.0.1	TCP	44	8000 → 65262 [ACK] Seq=1 Ack=575
10	0.002305400	127.0.0.1	127.0.0.1	TCP	169	8000 → 65262 [PSH, ACK] Seq=1 Ack=1
11	0.002326600	127.0.0.1	127.0.0.1	TCP	44	65262 → 8000 [ACK] Seq=575 Ack=1
12	0.002372600	127.0.0.1	127.0.0.1	HTTP/1.1	79	HTTP/1.1 200 OK, JSON (application/json)
13	0.002381100	127.0.0.1	127.0.0.1	TCP	44	65262 → 8000 [ACK] Seq=575 Ack=1
14	5.010611200	127.0.0.1	127.0.0.1	TCP	44	8000 → 65262 [FIN, ACK] Seq=161
15	5.010636600	127.0.0.1	127.0.0.1	TCP	44	65262 → 8000 [ACK] Seq=575 Ack=1
16	8.243117100	127.0.0.1	127.0.0.1	TCP	44	65262 → 8000 [FIN, ACK] Seq=575

C. Operación de Actualización (PATCH) Se ha validado la modificación parcial de los recursos mediante el método PATCH.

- Acción: Envío de una solicitud apuntando al recurso específico (path parameter) junto con el nuevo valor de ttl mediante query parameter.
- Resultado: Recepción de código 200 OK, confirmando la actualización del TTL en el registro persistido en memoria. Se ha verificado adicionalmente la gestión de errores mediante códigos 404 ante intentos de actualización de registros inexistentes.

*Adapter for loopback traffic capture						
Archivo Edición Visualización Ir Captura Analizar Estadísticas Telefonía Wireless Herramientas Ayuda						
tcp.port == 8000						
No.	Time	Source	Destination	Protocol	Length	Info
3	12.199925300	127.0.0.1	127.0.0.1	TCP	44	57960 → 8000 [FIN, ACK] Seq=1 Ack=1 Win=0 Len=0
4	12.199976100	127.0.0.1	127.0.0.1	TCP	44	8000 → 57960 [ACK] Seq=1 Ack=2 Win=256 Len=0
5	12.200496300	127.0.0.1	127.0.0.1	TCP	56	51950 → 8000 [SYN] Seq=0 Win=65535 Len=0
6	12.200565400	127.0.0.1	127.0.0.1	TCP	56	8000 → 51950 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0
7	12.200588700	127.0.0.1	127.0.0.1	TCP	44	51950 → 8000 [ACK] Seq=1 Ack=1 Win=0 Len=0
8	12.200808200	127.0.0.1	127.0.0.1	HTTP	670	PATCH /dns/practica3.com?nuevo_t...
9	12.200826000	127.0.0.1	127.0.0.1	TCP	44	8000 → 51950 [ACK] Seq=1 Ack=627 Win=65535 Len=0
12	12.202925100	127.0.0.1	127.0.0.1	TCP	169	8000 → 51950 [PSH, ACK] Seq=1 Ack=1 Win=65535 Len=0
13	12.202953200	127.0.0.1	127.0.0.1	TCP	44	51950 → 8000 [ACK] Seq=627 Ack=1 Win=0 Len=0
14	12.203006700	127.0.0.1	127.0.0.1	HTTP/1.1	73	HTTP/1.1 200 OK, JSON (application/...
15	12.203016800	127.0.0.1	127.0.0.1	TCP	44	51950 → 8000 [ACK] Seq=627 Ack=1 Win=0 Len=0

D. Operación de Eliminación (DELETE) Se ha verificado la eliminación de registros mediante el método DELETE, garantizando la limpieza de la base de datos en memoria.

- Acción: Solicitud de eliminación del recurso identificado por su nombre (path parameter).
- Resultado: Recepción de código 204 No Content, indicando que la operación se ha ejecutado exitosamente y que no existe contenido adicional que devolver al cliente. Asimismo, se ha comprobado la robustez del sistema ante la eliminación de recursos no encontrados, recibiendo el código 404 Not Found.

*Adapter for loopback traffic capture						
Archivo Edición Visualización Ir Captura Analizar Estadísticas Telefonía Wireless Herramientas Ayuda						
tcp.port == 8000						
No.	Time	Source	Destination	Protocol	Length	Info
9	3.825750000	127.0.0.1	127.0.0.1	TCP	44	58703 → 8000 [FIN, ACK] Seq=1 Ack=1 Win=0 Len=0
10	3.825794500	127.0.0.1	127.0.0.1	TCP	44	8000 → 58703 [ACK] Seq=1 Ack=2 Win=256 Len=0
11	3.826292000	127.0.0.1	127.0.0.1	TCP	56	59796 → 8000 [SYN] Seq=0 Win=65535 Len=0
12	3.826364500	127.0.0.1	127.0.0.1	TCP	56	8000 → 59796 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0
13	3.826387900	127.0.0.1	127.0.0.1	TCP	44	59796 → 8000 [ACK] Seq=1 Ack=1 Win=65535 Len=0
14	3.826601900	127.0.0.1	127.0.0.1	HTTP	644	DELETE /dns/practica3.com HTTP/1.1
15	3.826617600	127.0.0.1	127.0.0.1	TCP	44	8000 → 59796 [ACK] Seq=1 Ack=601 Win=65535 Len=0
18	3.829173400	127.0.0.1	127.0.0.1	HTTP	157	HTTP/1.1 204 No Content
19	3.829205100	127.0.0.1	127.0.0.1	TCP	44	59796 → 8000 [ACK] Seq=601 Ack=114 Win=65535 Len=0
22	8.830415100	127.0.0.1	127.0.0.1	TCP	44	8000 → 59796 [FIN, ACK] Seq=114 Ack=601 Win=0 Len=0
23	8.830438900	127.0.0.1	127.0.0.1	TCP	44	59796 → 8000 [ACK] Seq=601 Ack=115 Win=0 Len=0

C. Gestión de Errores

Se ha testado la robustez del servidor ante consultas de recursos inexistentes.

- Resultado: Recepción de código 404 Not Found, cumpliendo con la semántica REST ante errores de cliente.

4. Análisis Técnico: RMI vs. REST

La implementación de esta práctica permite concluir los siguientes beneficios del paradigma REST sobre RMI:

1. Visibilidad: El uso de herramientas de inspección de red (Wireshark) permite auditar las transacciones en texto plano, facilitando el debugging.
2. Stateless: Cada petición HTTP es independiente. A diferencia de RMI, donde el cliente mantenía una referencia al objeto remoto, en REST el servidor no necesita persistir el estado de la conexión, permitiendo una mayor escalabilidad.
3. Independencia del Lenguaje: Mientras que RMI está fuertemente acoplado a la JVM, este servicio DNS puede ser consumido por cualquier cliente capaz de interpretar HTTP y JSON (navegadores, cURL, aplicaciones móviles, etc.).
4. Gestión de estado: Mientras que la lógica de negocio (el almacenamiento de registros DNS) reside en memoria, el protocolo HTTP se mantiene stateless; el servidor no asume conocimiento previo sobre el cliente, facilitando la escalabilidad horizontal. Diseño de URLs: Se ha priorizado el uso de nombres de recursos en los endpoints (ej. /dns) en lugar de verbos, delegando la acción semántica a los métodos HTTP (POST, GET, PATCH, DELETE), conforme a los principios RESTful.

Método distribuido (Práctica 2)	Endpoint REST	Método HTTP
registrarDNS(nombre,ip,TTL)	/dns	POST
consultarDNS(nombre)	/dns/{nombre}	GET
actualizarTTL(nombre,TTL)	/dns/{nombre}	PATCH
eliminarRegistro(nombre)	/dns/{nombre}	DELETE

Servicio DNS RESTful (0.1.0)

Download OpenAPI specification: [Download](#)

Añadir registro

REQUEST BODY SCHEMA: `application/json`

`required`

<code>nombre</code> <code>required</code>	string (Nombre)
<code>ip</code> <code>required</code>	string (Ip)
<code>ttd</code> <code>required</code>	integer (Ttd)

Responses

> **201** Successful Response

> **422** Validation Error

5. Conclusión

La implementación de un servicio DNS bajo arquitectura REST ha demostrado ser más eficiente en términos de mantenibilidad y transparencia que el modelo RMI previo. El uso de validadores de esquema (Pydantic) y protocolos estándar (HTTP) asegura que el sistema sea interoperable y cumpla con las expectativas de robustez de los sistemas distribuidos modernos.

José Sánchez Bueno y Sergio Mozas Blanca